ZCAS UNIVERSITY

MASTER OF SCIENCE IN COMPUTER SCIENCE

CCS 6201

SOFTWARE METRICS

DEFERRED EXAMINATION

WEDNESDAY 18th JUNE 2025

TIME: 14.00 – 17.00 HOURS

TIME ALLOWED: THREE HOURS (plus 5 minutes to read through the paper)

INSTRUCTIONS

1. Section A: Question One in Section A is **compulsory**.
2. Sections B: Answer Two (2) questions from this section.
3. This examination paper carries a total of **100 marks**.
4. Candidates must **not turn this page** until the invigilator tells them to do so.

PAGE INTENTIONALLY

LEFT BLANK

SECTION A: QUESTION 1 IS COMPULSORY AND MUST BE ATTEMPTED

QUESTION 1:

Scenario: Read the scenario in Appendix A on **EduTech University Management System (UMS),** (also sent to you earlier), and then answer the following questions:

a) Explain each metric in the CK metrics suite and explain their relevance to object-oriented software quality. (12 Marks)

b) Distinguish *GradeEvaluator* and *PaymentProcessor* in terms of testability using RFC and LCOM metrics. (6 Marks)

c) Calculate the **average WMC** for all classes in the system. Explain what this indicates about overall class complexity? (4 Marks)

d) Explain the role of Object-Oriented Metrics in software quality assurance. Mention at least four quality attributes they can assess. (18 Marks)

e) Given weekly updates by multiple teams, discuss the metric and practice that should be enforced to manage code stability? (10 Marks)

[TOTAL: 50 MARKS]

SECTION B: ATTEMPT ONLY TWO QUESTIONS FROM THIS SECTION

QUESTION 2:

Use the data in the case study above (for Question 1.) on **EduTech University Management System** to answer the following questions;

a) From the CK metrics table, identify and explain which class may present the greatest maintainability challenge. Justify your answer using at least three metrics. (8 Marks)

b) Explain how the MOOD metrics MHF and AHF contribute to encapsulation and why these are desirable in object-oriented design. (7 Marks)

c) Discuss how static analysis tools complement metric-based evaluation?(10 Marks)

[TOTAL: 25 MARKS]

QUESTION 3:

Case Study: Model-Based Estimation for Satellite Software at AeroCompute Inc.

Background

AeroCompute Inc. is developing mission-critical satellite software for a government aerospace client. To manage cost and schedule risks, the company has decided to move away from expert judgment alone and implement **model-based estimation** using historical data, regression models, and cost estimation relationships (CERs).

They have selected a parametric model that includes inputs such as:

- Software size (in KSLOC)
- Complexity level (Low, Medium, High)
- Development environment capability
- Team experience
- Reuse percentage
- Required reliability

The project involves approximately **150 KSLOC**, with **medium complexity, moderate team experience, 20% reuse**, and **high reliability requirements**. The estimation model is based on the following simplified form:

Effort (in person-months) = A × (KSLOC)^B × EM

Where:

- A = 2.5
- B = 1.05
- EM (Effort Multiplier) is calculated as a product of multiple cost drivers. For this case, EM = 1.20 due to high reliability and moderate experience.

QUESTIONS:

a) Explain how increasing software reuse from 20% to 50% might affect the model-based estimate and what implications it has for planning. (10 Marks)

b) Discuss two limitations or risks of model-based estimation in the context of AeroCompute's project. (5 Marks)

c) Recommend two practices AeroCompute should adopt to continuously improve the accuracy of their model-based estimates over time. (10 Marks)

[TOTAL: 25 MARKS]

QUESTION 4:

Case Study Title: "Comparative Software Sizing and Effort Estimation in the DevSync ERP Project"

Background:

DevSync Technologies is developing an Enterprise Resource Planning (ERP) solution for SMEs. The system supports modules such as:

- **Inventory Management**
- **Sales and Customer Relations**
- **Finance and Accounting**
- **HR and Payroll**

The project is developed using a mix of **Java (backend), Angular (frontend),** and **PostgreSQL (database).**

To plan effort, cost, and project timelines, the company conducted **software size and productivity evaluation** using multiple metrics:

1. **Lines of Code (LOC)**
2. **Function Point Analysis (FPA)**
3. **COCOMO II Model**
4. **Expert Judgment and Analogy-based estimation**

Metric Data from the Inventory and Sales Modules:

Inventory Module (Java + PostgreSQL):

Metric	Value
Total Source Lines of Code	9,000
Number of Internal Inputs	12
Number of External Outputs	8
Number of External Inquiries	6
Number of Internal Logical Files	5
Number of External Interface Files	3
Adjusted Complexity Weight	1.2

Sales Module (Java + Angular):

Metric	Value
Total Source Lines of Code	12,500
Number of Internal Inputs	15
Number of External Outputs	10
Number of External Inquiries	8
Number of Internal Logical Files	6
Number of External Interface Files	4
Adjusted Complexity Weight	1.3

Other information:

- Productivity rate: 10 FP per person-month
- Cost per person-month: USD 5,000
- Schedule pressure is high; two-month delivery for both modules is expected
- Code reuse is estimated at 20% for each module
- Maintenance is projected at 25% of initial development effort over two years

QUESTIONS:

a) Differentiate between Lines of Code (LOC) and Function Points (FP) as software size measures. Provide one advantage and one limitation of each. (12 Marks)

b) Calculate the Unadjusted Function Points (UFP) for the Inventory Module using standard weights: (6 Marks)

c) Name and briefly describe three effort estimation techniques used in practice besides LOC and FP. (7 Marks)

[TOTAL: 25 MARKS]

APPENDIX A

Case Study Title: "Evaluating Object-Oriented Quality in the EduTech University Management System (UMS)"

Background:

EduTech Solutions, a software development firm, was contracted to build a large-scale **University Management System (UMS)** a consortium of universities. The UMS was designed using **Object-Oriented Programming (OOP)** principles and implemented in **Java**

The system supports:

- Student registration, course management, and grading
- Staff records and payroll
- Library borrowing and fee management
- Scheduling, timetabling, and reporting

To ensure the quality of the software system, EduTech decided to evaluate the system using object-oriented metrics related to **maintainability**, **reusability**, **understandability**, and **testability**. The firm applied both **CK Metrics Suite (Chidamber and Kemerer)** and **MOOD Metrics (Metrics for Object-Oriented Design)** to perform quantitative evaluation of the UMS codebase.

System Design Overview:

- Implemented using **Model-View-Controller (MVC)** architecture
- Key design patterns applied: **Singleton**, **Observer**, and **Factory**
- Over 90 classes in total; 5 key subsystems examined here

The **metrics evaluation** was done using:

1. **CK Metrics Suite**
2. **MOOD Metrics**
3. **Static Code Analysis Tools** (e.g., SonarQube)
4. **Qualitative inspection of design patterns and documentation**

A sample of Selected Metric Results for Five Key Classes from the analysis is provided below:

Class Name	WMC	DIT	NOC	CBO	RFC	LCOM
StudentManager	20	2	0	12	28	3
CourseManager	15	1	0	10	22	2
PaymentProcessor	30	3	1	15	35	5
GradeEvaluator	10	2	0	8	15	1
ReportGenerator	25	1	0	18	33	4

MOOD Metrics Summary:

- **Method Hiding Factor (MHF):** 0.78
- **Attribute Hiding Factor (AHF):** 0.82
- **Method Inheritance Factor (MIF):** 0.35
- **Attribute Inheritance Factor (AIF):** 0.30
- **Polymorphism Factor (PF):** 0.45
- **Coupling Factor (CF):** 0.28

Additional Evaluation Factors:

- **Code Smells Identified**:
  - *God Class* in ReportGenerator
  - *Long Method* in PaymentProcessor
  - *Shotgun Surgery* in StudentManager
- **Static Analysis Warnings**:
  - High cyclomatic complexity (>10) in 8 methods
  - 20% of methods lack Javadoc comments
  - Unused variables and dead code in 12 classes
- **Evolution Concerns**:
  - System maintenance frequency is high
  - Several modules tightly coupled and fragile to change
  - Codebase updated weekly by multiple distributed teams

Objective:

The UMS project manager has tasked you with evaluating and interpreting these metrics to determine the system's quality in terms of object-oriented quality attributes such as maintainability, testability, reusability, and complexity. You are also required to suggest improvements where necessary.

END OF CASE STUDY

END OF EXAMINATION